Cyber Security

Project

Name : Chandana Somanath Khatvakar

Project Title: Stored Cross-Site Scripting (XSS)

Introduction:

Stored Cross-Site Scripting (XSS) is a type of web security vulnerability where an attacker injects malicious scripts into a website's database or stored data. These scripts are later served to users, compromising their sessions, stealing sensitive information, or performing other malicious actions. Since the script is stored on the server, it impacts all users who access the affected page.

Here's a quick overview of how stored XSS works:

1. **Injection of Malicious Script**: The attacker submits a malicious script to a site, often via form inputs like comments, profile settings, or message boards. If the site doesn't properly validate or sanitize input, it saves the script in its database.
2. **Script Execution**: When other users access the page containing the stored script, their browsers execute it as part of the page content. This is especially dangerous since the browser treats the script as coming from a trusted source.
3. **Potential Impacts**: The script can steal cookies, impersonate the user, log keystrokes, or redirect to malicious sites, among other actions.
4. **Prevention**: Proper input validation and output encoding, along with security measures like Content Security Policy (CSP), are crucial in preventing stored XSS vulnerabilities.

Stored XSS is considered one of the more severe XSS types due to its persistence and impact on multiple users.

- **Advantages of Stored XSS (from an attacker's perspective)**

1. **Persistent Payload**: Once injected, the malicious script stays in the system and affects multiple users without further action from the attacker.
2. **Wider Reach**: Since the payload is stored on the server, it can potentially reach all users who visit the affected page, amplifying the attack's impact.
3. **Stealth**: Stored XSS can be difficult for users to detect since the script runs as part of the legitimate website, often making it appear trustworthy.
4. **Automation Potential**: Attackers can automate the extraction of stolen data or account information from all affected users with minimal effort after the initial injection.

- **Disadvantages of Stored XSS (from a security and business perspective)**

1. **User Data Exposure**: Stored XSS can compromise users' private data, including session tokens and personal information, leading to privacy breaches and potential data leaks.

2. **Reputational Damage**: A successful XSS attack can harm a company's reputation, as users lose trust in the site's security measures.
3. **Legal and Compliance Risks**: Data breaches can lead to legal consequences and non-compliance fines, especially for sites that handle sensitive user data.
4. **Resource-Intensive Remediation**: Fixing stored XSS issues may require significant developer time and effort, including sanitizing data, patching vulnerabilities, and auditing stored data to ensure other instances aren't affected.
5. **Financial Loss**: Attacks can lead to direct financial losses if user accounts or payment systems are exploited and might drive users away from the service.

In summary, stored XSS is advantageous to attackers due to its persistence and broad impact but poses significant risks and costs to both users and businesses.

- ## Procedure:

Here's a typical **procedure for a stored XSS attack** and steps to **prevent it**:

Attack Procedure for Stored XSS

1. **Identify Input Points**: An attacker finds a user input field on a web application where they can submit data that gets stored and displayed to other users. Examples include comment sections, profile fields, or message boards.
2. **Inject Malicious Script**: The attacker inputs a malicious JavaScript payload into one of these fields. A simple example could be:

   ```
   javascript
   Copy code
   <script>alert('XSS')</script>
   ```

   or more sophisticated scripts to steal cookies or redirect users to phishing pages.

3. **Server-Side Storage**: If the application doesn't validate or sanitize the input, the script is stored in the database or application backend.
4. **Script Execution on Page Load**: When another user views the page containing the injected script, their browser loads and executes the script. Since it originates from the trusted site's domain, it bypasses many security checks.
5. **Attacker Gains Access**: The attacker can gain unauthorized access to sensitive data, hijack user sessions, perform actions on behalf of users, or redirect them to malicious sites.

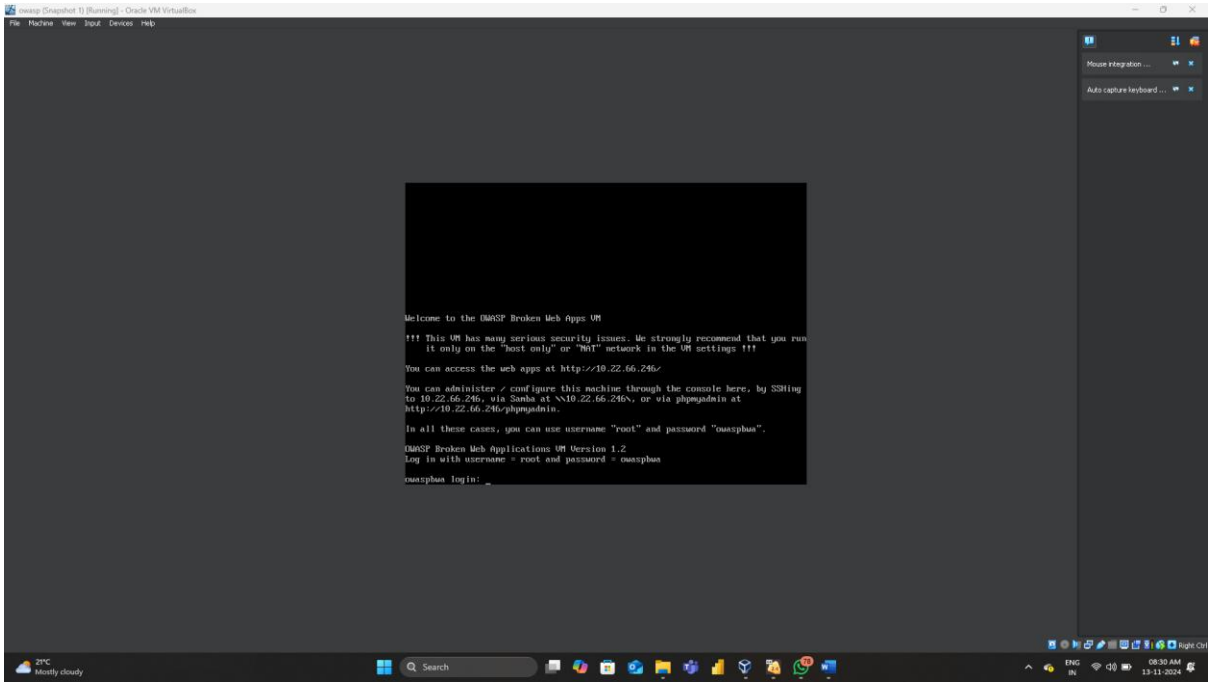
Prevention Steps for Stored XSS

1. **Input Validation and Sanitization**: Filter and validate all user inputs on the server side. Reject any inputs containing potentially dangerous characters (e.g., `<`, `>`, `&`, `"`).
2. **Output Encoding**: When displaying data to users, ensure the data is properly encoded to prevent it from being interpreted as executable code. For example, encode special characters so < and > appear as `<` and `>`.

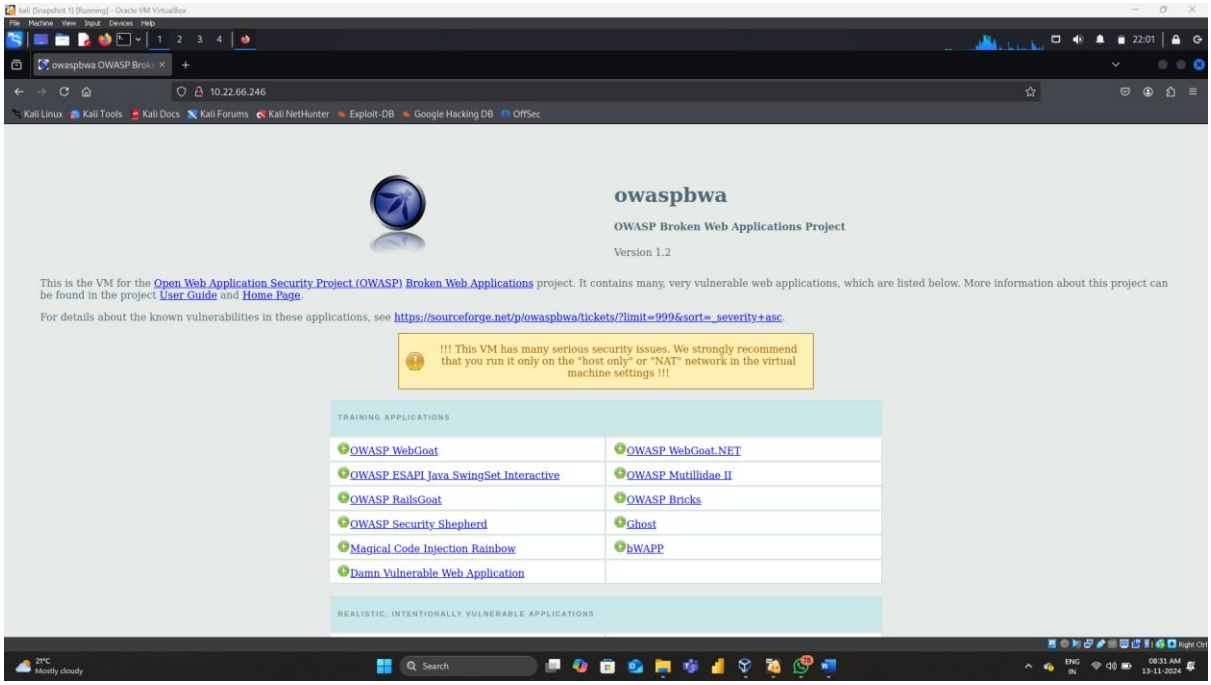
3. **Use a Web Application Firewall (WAF)**: A WAF can help detect and block malicious requests before they reach the server, offering an extra layer of protection.
4. **Content Security Policy (CSP)**: Implement a strict CSP to control which scripts can execute on your site. CSP can prevent malicious scripts from running, even if they are injected.
5. **Implement HTTP-Only and Secure Cookies**: By setting cookies with the `HttpOnly` and `Secure` attributes, you limit access to cookies from JavaScript, reducing the risk of session hijacking.
6. **Regular Security Audits**: Regularly audit code and data storage for potential vulnerabilities and ensure all developers are trained in secure coding practices.

Following these procedures helps prevent stored XSS vulnerabilities, reducing risk to both users and the application.

Step 1: start OWASP machine

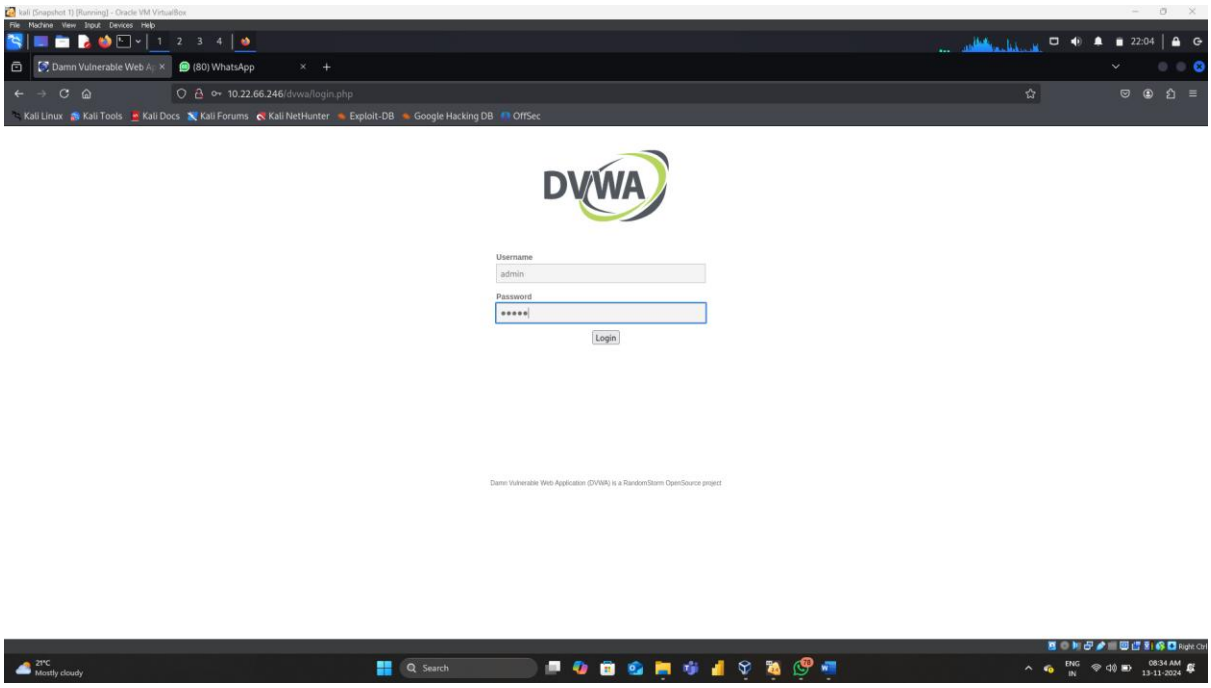


Step 2: On Kali Linux open abrowser and give the ip adress of OWASP machine

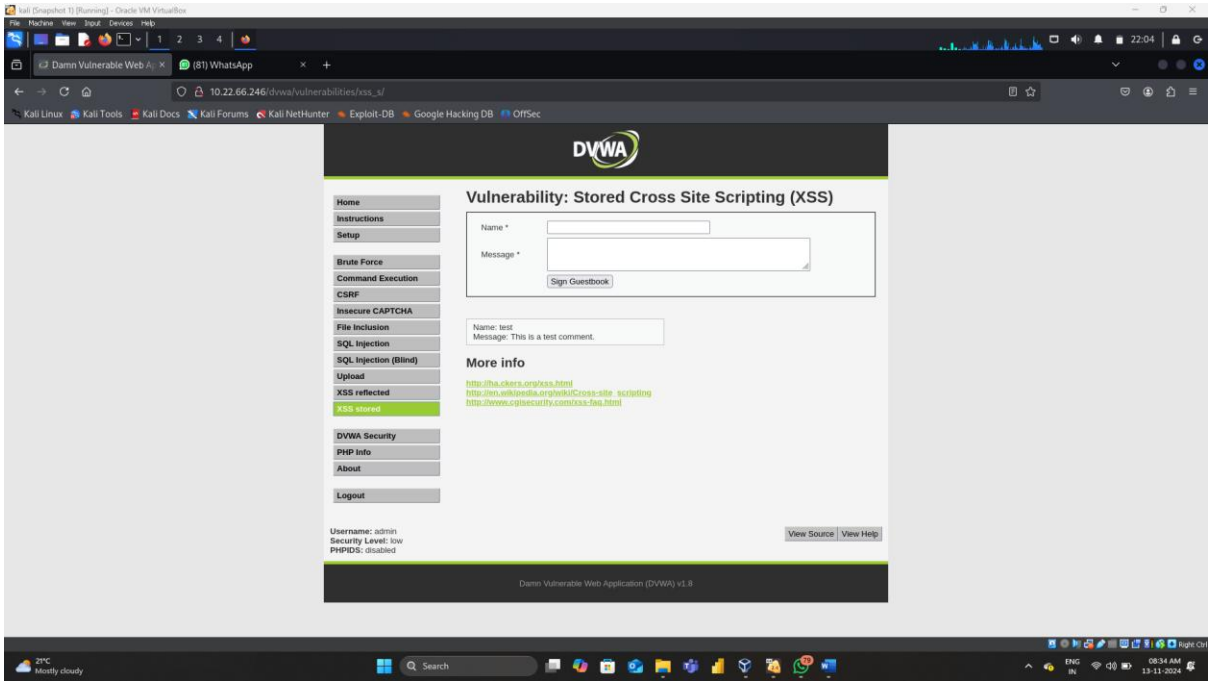


Step 3: Click on Damn Vulnarable Web Application

Step 4: A login page will display give your username and password

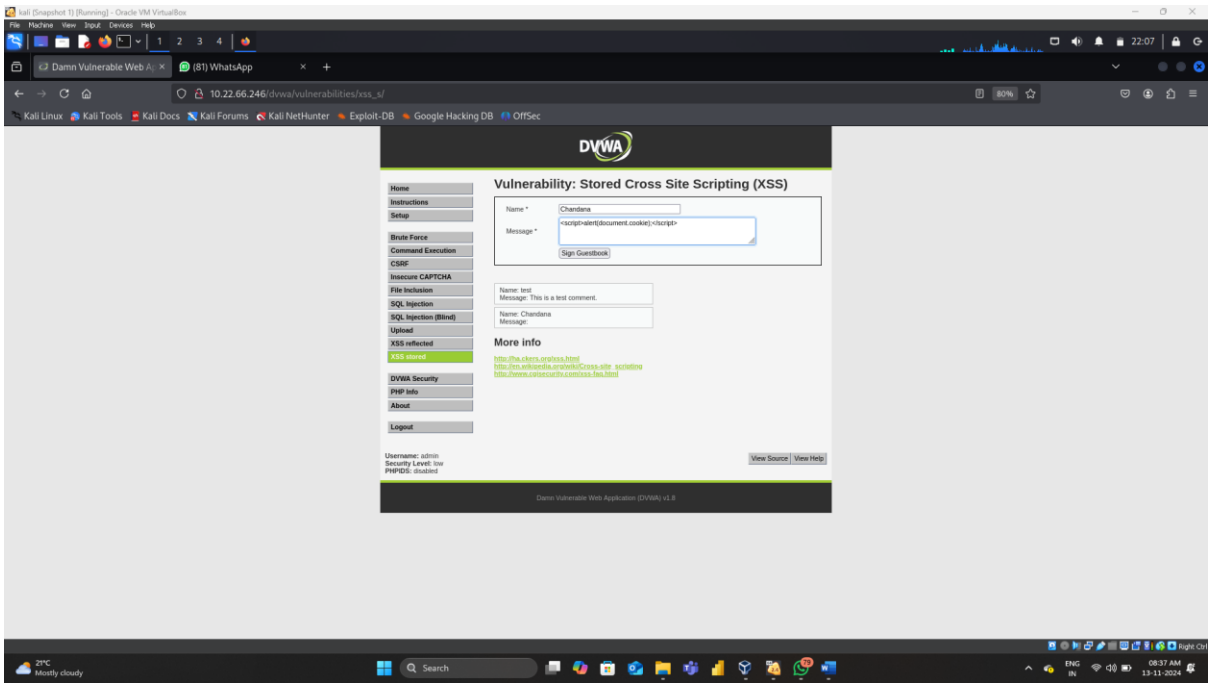


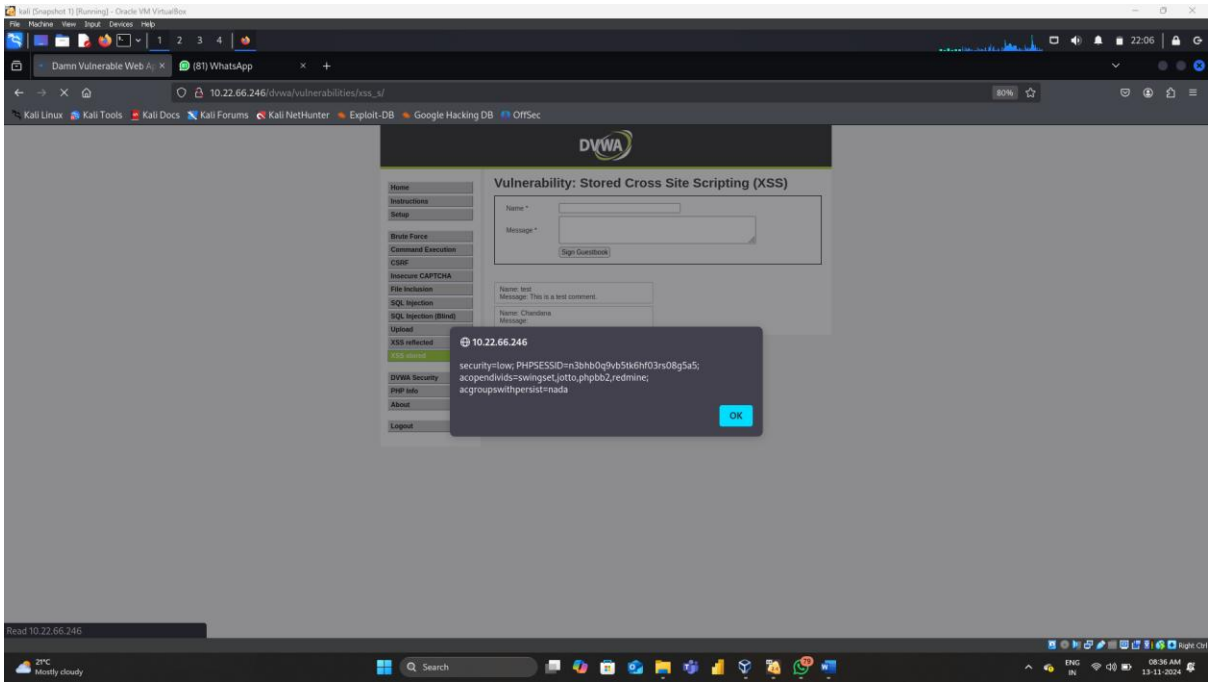
Step 5:Click on XSS stored



Step 6: Give name as bhavani

Message as <script>alert(document.cookie);</script> and click on sign guestbook




Damn Vulnerable Web A
(81) WhatsApp
10.22.66.246/dvwa/vulnerabilities/xss_s/
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OffSec
DVWA
Vulnerability: Stored Cross Site Scripting (XSS)
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
Insecure CAPTCHA
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
DVWA Security
PHP Info
About
Logout
Name *
Message *
Sign Guestbook
Name: test
Message: This is a test comment.
Name: Chandana
Message:
10.22.66.246
security=low; PHPSESSID=n3bhb0q9vb5tk6hf03rs08g5a5;
acopendivids=swingset,jotto,phpbb2,redmine;
acgroupswithpersist=nada
OK
Read 10.22.66.246
Search